

Real-Time Prediction of Docker Container Resource Load Based on a Hybrid Model of ARIMA and Triple Exponential Smoothing

Yulai Xie[✉], *Member, IEEE*, Minpeng Jin, Zhuping Zou[✉], Gongming Xu[✉],
Dan Feng, *Member, IEEE*, Wenmao Liu, and Darrell Long[✉], *Fellow, IEEE*

Abstract—More and more enterprises are beginning to use Docker containers to build cloud platforms. Predicting the resource usage of container workload has been an important and challenging problem to improve the performance of cloud computing platform. The existing prediction models either incur large time overhead or have insufficient accuracy. This article proposes a hybrid model of the ARIMA and triple exponential smoothing. It can accurately predict both linear and nonlinear relationships in the container resource load sequence. To deal with the dynamic Docker container resource load, the weighting values of the two single models in the hybrid model are chosen according to the sum of squares of their predicted errors for a period of time. We also design and implement a real-time prediction system that consists of the collection, storage, prediction of Docker container resource load data and scheduling optimization of CPU and memory resource usage based on predicted values. The experimental results show that the predicting accuracy of the hybrid model improves by 52.64, 20.15, and 203.72 percent on average compared to the ARIMA, the triple exponential smoothing model and ANN+SaDE model respectively with a small time overhead.

Index Terms—Docker container, prediction, hybrid model

1 INTRODUCTION

WITH the development and popularization of cloud computing platforms, most enterprises have their own data centers. By providing users with various virtual resources, such as computing resources, storage resources and network resources, users can get high quality, strong security and highly scalable infrastructure services at relatively low cost [1]. However, with the continuous expansion of the cloud computing platform, the virtual machine [2] has problems such as low running efficiency and slow start-up. To alleviate these problems, Docker container [3] has emerged as a new virtualization technology.

Whether it's in the virtual machine or Docker container, we should allocate sufficient resources for an application to run smoothly. However, in most cases, the application is not running at the heaviest load, the pre-provisioned resources are idle most of the time. This causes a waste of resources. In

addition, when the workload is heavy and has to compete with other applications simultaneously, the pre-allocated resources may be not enough. In order to solve such problems, a specific prediction algorithm is usually used in cloud computing to predict resource requirements, and resource allocation optimization is performed in advance to improve resource utilization and service quality.

At present, there are few related studies on resource load prediction of Docker containers. Shanmugam *et al.* [4] predicted the CPU usage of the container through ARIMA model and then distributed the load to the container's web service using a loop-based algorithm. Roy *et al.* [5], [6] proposed a cloud computing load prediction model based on ARIMA model, which smooths the time series first. Huang *et al.* [7] proposed a resource prediction model based on quadratic exponential smoothing to predict the cloud resources that customers need to subscribe to. It not only considers the current resource status but also considers historical resource records and obtains higher prediction accuracy.

The above several types of resource load predictions are based on ARIMA model or the quadratic exponential smoothing model. This is because the resource load sequence is a time series, and the two models are common prediction models for time series prediction. But whether it is ARIMA model or quadratic exponential smoothing model, they in essence are linear models. However, the time series generated by different resource loads in the Docker container are not only linear but also non-linear, as shown in Fig. 1. These two models do not have a very good prediction accuracy on predicting the nonlinear relationship in the container resource load sequence.

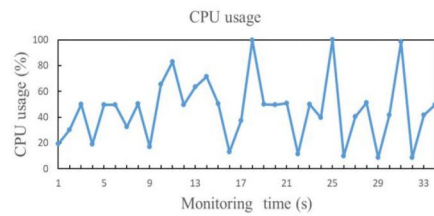
- Y. Xie is with Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, P.R. China. E-mail: ylxie@hust.edu.cn.
- M. Jin, Z. Zou, G. Xu, and D. Feng are with the School of Computer, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, P.R. China. E-mail: {jinminpeng0510, zouzhup, xugongming38}@gmail.com, dfeng@hust.edu.cn.
- W. Liu is with NSFOCUS Inc., Haidian District, Beijing 100089, China. E-mail: liuwenmao@nsfocus.com.
- D. Long is with the Jack Baskin School of Engineering, University of California, Santa Cruz, CA 95064 USA. E-mail: darrell@ucsc.edu.

Manuscript received 25 Sept. 2019; revised 23 Feb. 2020; accepted 18 Apr. 2020.
Date of publication 22 Apr. 2020; date of current version 7 June 2022.

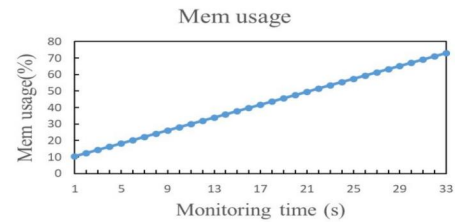
(Corresponding author: Yulai Xie.)

Recommended for acceptance by B. Schulze.

Digital Object Identifier no. 10.1109/TCC.2020.2989631



a. CPU usage of Docker container load



b. Memory usage of Docker container load

Fig. 1. The linear and non-linear relationships in the Docker container resource load. The experiment data are acquired by using the stress (version 1.0.4) tool to test the resource usage of a memcached (version 1.5.6) container on the Ubuntu operating system.

However, though ARIMA can be used in both container and virtual machine, as the resource usage sequence jitters more in a container environment than in a virtual machine, ARIMA is more suitable for container environment to eliminate the random fluctuation of the container resource load sequence. In addition, as each container usually starts and closes in a short period, this will result in the failure to collect a large amount of container resource usage data. In the case of insufficient historical data, some machine learning prediction models (such as neural networks and linear regression prediction methods [8]) that have been used in virtual machines are not suitable for use in containers.

It can be seen that the existing models either cannot predict both the linear and non-linear workloads or are not suitable for use in the container environment. To address the above problems, this paper proposes a hybrid model of the ARIMA and triple exponential smoothing model to predict both the linear and non-linear relationships in the Docker container resource load. To deal with the dynamic Docker container resource load, the weight values of the two models in the hybrid model are chosen according to the sum of squares of their respective predicted errors for a period of time. In the process of prediction, ARIMA model is used to mine the linear relationship and eliminate the random fluctuation of the container resource load sequence, while triple exponential smoothing is used to mine the nonlinear relationship and smooth the container resource load sequence. We also design and implement a real-time prediction system for Docker container workload. The system implements the collection, storage, prediction of Docker container resource load data and scheduling optimization of CPU and memory resource usage based on predicted values.

The contributions of this paper are as follows:

- We propose a hybrid prediction model of ARIMA and triple exponential smoothing that can predict both linear and nonlinear relationships in the container load sequence and significantly improve the Docker container load prediction accuracy.
- We design a Docker container load prediction system that can predict multidimensional resource load, and automatically make schedule optimization of CPU and memory resource usage based on predicted values.
- We evaluate the hybrid model on a variety of Docker containers in both simulated and real cloud environments. The experimental results demonstrate the hybrid model has a good prediction accuracy with a small time overhead.

2 BACKGROUND AND MOTIVATION

We first introduce the Docker technology, then we describe the ARIMA and triple exponential smoothing model and then motivate our research.

2.1 Docker Technology

Docker is an open source application container engine that packages the application and its runtime environment into a lightweight, portable container [9], [10]. Docker can be built on top of LXC [11] or libcontainer [12]. Libcontainer controls containers by managing namespaces, cgroups, capabilities, and file systems. In order to ensure that the processes among the different containers do not interfere and affect each other, libcontainer isolates the resources they use through namespaces. In order to solve the problem of competing resources between containers, libcontainer also uses cgroups to limit and isolate the resource usage (CPU, memory, disk I/O, network, etc.). Compared with virtual machines, Docker container is more light-weight and enables quick creation and destruction [13].

With the popularity of Docker container technology, more and more Internet companies are using large container clusters to serve as application runtime environments, such as Amazon AWS, Microsoft Azure, and Alibaba Cloud, which already support Docker containers and also provide Container as a Service (CaaS) [14]. In order to facilitate the management of Docker clusters, we can use Docker swarm or Kubernetes to easily deploy container clusters of multiple nodes [15], [16].

2.2 ARIMA Model

The ARIMA(p, d, q) model [17] is called the differential autoregressive moving average model that predicts time series using a linear combination of AR model (autoregressive model) and MA model (moving average model). p and q are the orders of the autoregressive model and moving-average model respectively, and d is the number of differences required to make the original time series into a stationary sequence.

When applying ARIMA model to container resource load prediction, it can be roughly divided into the following steps:

- 1) collecting and obtaining a time series of container resource usage.
- 2) ADF test [18], which is called augmented dickey-fuller test, is used to determine whether the time series is stable. If it is not stable, the time series is processed by using difference until the series is stable. The times of difference is recorded as d .

- 3) Obtain the range of values of the order of the autoregressive part p and the order of the moving-average part q of the stationary sequence. The different p and q values are substituted into the model to get the AIC values [19]. And the values of p and q with the smallest AIC value determine the best ARIMA model for this time series.
- 4) The autocorrelation coefficient and partial autocorrelation coefficient are calculated by using Levinson algorithm [20].
- 5) Then the ARIMA model can be used to predict future resource use of the container.

The resource usage value y_t at each moment of the container is expressed as the linear function of the container resource usage values of the previous p times, the prediction error value of the container resource usage values of the previous q times and the error term at current time. The linear function formula is as follows:

$$y_t = \mu + \varepsilon_t + \sum_{i=1}^p \gamma_i y_{t-i} + \sum_{i=1}^q \theta_i \varepsilon_{t-i}. \quad (1)$$

μ is constant term. ε is the prediction error sequence of the model. p is the autoregressive number. q is the moving average number. γ is the autocorrelation coefficient, that is, the respective weights of the container resource usage values of the previous p times. θ is the partial autocorrelation coefficient, that is, the respective weights of the prediction error values of the container resource usages of the previous q times.

However, ARIMA model cannot mine and grasp the nonlinear trend of a time series. When applying ARIMA model to the prediction of container load resources, it will not get great accuracy. The best performance of the algorithm for approximating the nonlinear trend in the sequence is the artificial neural network algorithm. But for small-scale cloud computing platform built with Docker containers, the artificial neural network algorithm consumes too much computing and storage resource. Therefore, artificial neural networks are often used for large-scale cluster resource usage prediction. Moreover, the training period of artificial neural networks tends to be too long and requires a large amount of data, which is an unacceptable overhead in the real-time prediction of the container resource load. So we choose triple exponential smoothing to mine the nonlinear trend of time series.

2.3 Triple Exponential Smoothing

Exponential smoothing method [21] uses a special weighted averaging method to achieve the smoothing of the time series data samples. Its principle is to decompose the time series into three parts: the overall mean, the overall trend, and the seasonal trend. It assigns weights in a unique way, that is, the more distant the time point of historical data is from the current time point, the less weight is given to the true value of the time point. The true value before the current time point is given a weight that decreases exponentially from near to far and gradually converges to zero. This not only ensures the integrity of the time series information but also focuses on the information at different points in time.

Triple exponential smoothing model [22] applies exponential smoothing three times. There are generally three

steps: the first step is to calculate the smoothed values using the current value of the current time point and the smoothed values of the previous time point; the second step is to use the three smoothed values obtained in the first step to obtain the coefficients of the prediction model; the third step is to use the obtained coefficients to establish a mathematical model for prediction. The smoothed values of triple exponential smoothing model are as follows:

$$S_t^{(1)} = \alpha x_t + (1 - \alpha)S_{t-1}^{(1)} \quad (2)$$

$$S_t^{(2)} = \alpha S_t^{(1)} + (1 - \alpha)S_{t-1}^{(2)} \quad (3)$$

$$S_t^{(3)} = \alpha S_t^{(2)} + (1 - \alpha)S_{t-1}^{(3)}, \quad (4)$$

α is the smoothing factor, $S_t^{(1)}$ is the smoothed value of an exponential smoothing model, x_t is the observed value of the current time point, $S_t^{(2)}$ is the smoothed value of the quadratic exponential smoothing model, and $S_t^{(3)}$ is the smoothed value of triple exponential smoothing model. After obtaining the above three smoothed values, the coefficients of triple exponential smoothing model can be calculated as follows:

$$a_t = 3S_t^{(1)} - 3S_t^{(2)} + S_t^{(3)} \quad (5)$$

$$b_t = \left(\frac{\alpha}{2(1 - \alpha)^2} \right) \left[(6 - 5\alpha)S_t^{(1)} - 2(5 - 4\alpha)S_t^{(2)} + (4 - 3\alpha)S_t^{(3)} \right] \quad (6)$$

$$c_t = \left(\frac{\alpha^2}{2(1 - \alpha)^2} \right) \left[S_t^{(1)} - 2S_t^{(2)} + S_t^{(3)} \right]. \quad (7)$$

Then triple exponential smoothing model for prediction is obtained as Formula (8):

$$F_{t+m} = a_t + b_t m + c_t m^2. \quad (8)$$

m is the number of predicted points starting from the time point t . It can be seen that triple exponential smoothing model can mine and predict the nonlinear trend of time series. Each time a prediction is made, the resource usage of the container at the next time point can be predicted by using the data of each resource usage of the container at the current time point and the smoothed values of the previous time point.

However, triple exponential smoothing model is a nonlinear model, which cannot mine the linear relationship in the resource load sequence. In addition, when the weighted average smoothing of historical data is carried out by triple exponential smoothing model, the information of some other influencing factors contained in the data will be lost, and random fluctuations in the resource load sequence are not considered.

3 HYBRID MODEL BASED ON ARIMA AND TRIPLE EXPONENTIAL SMOOTHING

In this section, we first introduce the advantage of the hybrid model and then we elaborate the design of the model.

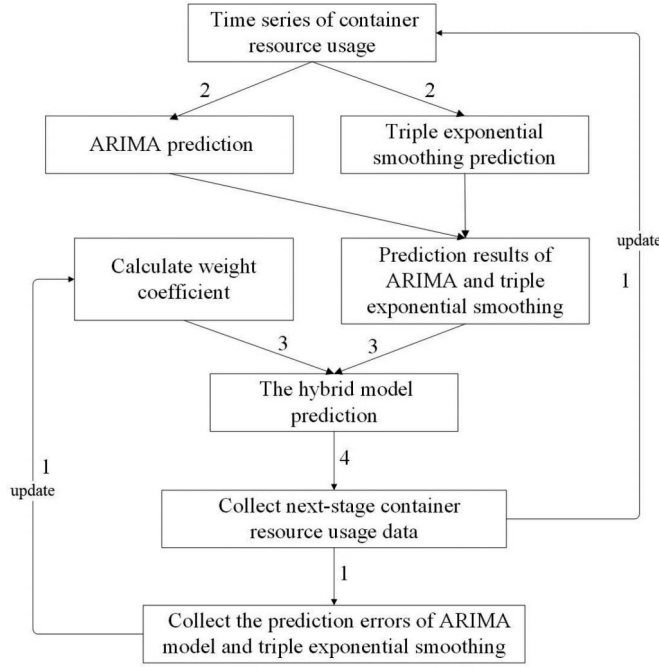


Fig. 2. The execution process of the hybrid model algorithm. The numbers on the line indicate the order of execution.

3.1 Advantage of the Hybrid Model

ARIMA model and triple exponential smoothing model have their own shortcomings when applied to the container resource load prediction. But their respective advantages can be complementary as follows.

First, ARIMA is a model to mine the linear relationship between different data items, while the triple exponential smoothing finds the nonlinear relationship buried in a large amount of data. Their combination can cope with a wide variety of workloads with different characteristics.

Second, ARIMA is mainly to dig the inherent relationship between data items, while the triple exponential smoothing further exploits the change trend of the whole time series. Their combination can describe the whole data characteristics more accurately.

Third, the hybrid model has a much stronger data anti-jamming ability. On one hand, MA model in the ARIMA model focuses on the accumulation of the predicted error terms. This can effectively eliminate the random fluctuations in the load sequence of container resources. On the other hand, the triple exponential smoothing model can assign different weights to historical data to smooth data. This makes the hybrid model has a much stronger anti-interference ability.

3.2 Design of the Hybrid Model

How to determine the weight coefficients of combined ARIMA and triple exponential smoothing is the key to the design of the hybrid model. Commonly used methods are the equal-weighted average coefficient method and weight coefficient determination method based on the error-index. The equal-weighted average coefficient method assigns the same weight to ARIMA and triple exponential smoothing. However, these two models have different prediction accuracy especially with the dynamics of container resource load. That is, it may occur in a certain time period, ARIMA model performs better, and triple exponential smoothing model

performs better in the next time period; or ARIMA predicts better on one application workload, and triple exponential smoothing predicts well on the other application. In order to better adapt to the dynamics of Docker container resources, we choose the weight coefficient determination method based on error-index as shown in Algorithm 1.

Since the error in the process of continuous prediction is also continuously generated, in order to make the weight coefficient keep up with the change of the time series of container resource usage, the weight of the model in the next prediction can be obtained by using the sum of squares of prediction error. Assume the size of the time series window is *size*. We accumulate the latest *size* prediction errors. If there is not *size* prediction errors, we use sum of the prediction errors that have been generated and the initial weight is 0.5. When the number of prediction errors generated is greater than *size*, every time the latest prediction error is obtained, the oldest prediction error is removed. In this way, the weight of the two single models can be adjusted with the change of time series of container resource usage.

Algorithm 1. calculate_weight

Input: Err_arima_t, Err_es_t // Err_arima_t is prediction error of ARIMA model at current time point and Err_es_t is prediction error of triple exponential smoothing model at current time point.

Output: $Weight_arima, Weight_es$ // $Weight_arima$ is the weight of ARIMA model next time and $Weight_es$ is the weight of triple exponential smoothing model next time.

```

1: if  $Err\_arima.size = size$  and  $Err\_es.size = size$  then
2:    $Sum\_arima \leftarrow Sum\_arima - Err\_arima[0]^2$ 
3:    $Sum\_es \leftarrow Sum\_es - Err\_es[0]^2$ 
4:   Remove the first element from array  $Err\_arima$  and array  $Err\_es$ 
5: end if
6:  $Err\_arima.push\_back(Err\_arima_t)$ 
7:  $Err\_es.push\_back(Err\_es_t)$ 
8:  $Sum\_arima \leftarrow Sum\_arima + Err\_arima_t^2$ 
9:  $Sum\_es \leftarrow Sum\_es + Err\_es_t^2$ 
10:  $Weight\_arima \leftarrow Sum\_es / (Sum\_arima + Sum\_es)$ 
11:  $Weight\_es \leftarrow Sum\_arima / (Sum\_arima + Sum\_es)$ 
12: return  $Weight\_arima, Weight\_es$ 
  
```

In Algorithm 1, Err_arima and Err_es are sliding windows, which are implemented with array, recording the prediction errors of ARIMA model and triple exponential smoothing model respectively in the previous period. Sum_arima and Sum_es are the sum of the prediction error squares of the two models respectively.

The execution process of the hybrid model algorithm can be divided into the following steps and is shown in Fig. 2.

- 1) Every time container resource usage data is collected, the time series of container resource usage is updated. Meanwhile, the prediction errors of ARIMA model and triple exponential smoothing model are calculated to update the weight coefficient of the hybrid model.
- 2) The updated time series are predicted by using ARIMA model and triple exponential smoothing model respectively.
- 3) According to the prediction results of ARIMA model and triple exponential smoothing model and the

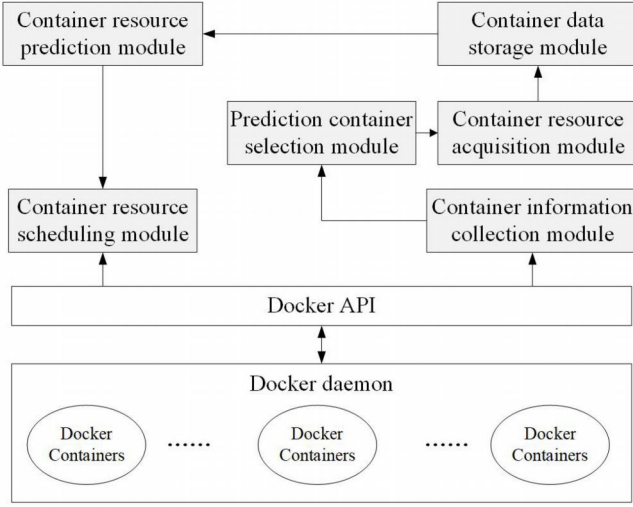


Fig. 3. System architecture.

previously determined weight coefficient, the prediction result of the hybrid model is sum of the prediction result of ARIMA model multiplied by the corresponding weight coefficient and the prediction result of triple exponential smoothing model multiplied by the corresponding weight coefficient.

- 4) After collecting the true value of the container resource usage data at the next time point, go back to step 1 to make the next prediction.

Algorithm 2. Hybrid Model Prediction Algorithm

Input: y_t // y_t is current observation value.

Output: *Predict_value* // The predicted value of the hybrid model.

```

1:  $Err\_arima_t \leftarrow y_t - Pred\_arima_t$ 
2:  $Err\_es_t \leftarrow y_t - Pred\_es_t$ 
3:  $(Weight\_es, Weight\_arima) \leftarrow$ 
    $calculate\_weight(Err\_es_t, Err\_arima_t)$ 
4:  $Pred\_arima_{t+1} \leftarrow ARIMA.predict(y_t)$ 
5:  $Pred\_es_{t+1} \leftarrow TripleExpSmoothing.predict(y_t)$ 
6:  $Pred\_value \leftarrow Pred\_arima_{t+1} \times Weight\_arima + Pred\_es_{t+1} \times$ 
    $Weight\_es$ 
7: return  $Pred\_value$ 
  
```

The algorithm is shown in Algorithm 2. $Pred_arima_t$ and $Pred_es_t$ represent the predicted value of the ARIMA model and triple exponential smoothing model respectively in a period. Err_arima_t and Err_es_t represent the prediction error of the ARIMA model and triple exponential smoothing model respectively. $Pred_arima_{t+1}$ and $Pred_es_{t+1}$ represent the predicted value of the ARIMA model and triple exponential smoothing model respectively in the next period. $Weight_arima$ and $Weight_es$ represent the weight of the ARIMA model and triple exponential smoothing model respectively.

Assume the length of the workload sequence of container resource is L . In ARIMA model, assume the value of the order of autoregressive model is p , the value of the order of the moving-average model is q , where the value range of p and q is $0 \leq p, q \leq N$, and p and q cannot be both zero. The main computational cost of ARIMA model algorithm is to determine the order of autoregressive model and the order of moving-average model and calculate the autocorrelation

coefficient and partial autocorrelation coefficient. Its time complexity is $O(Nq \ln L + L(p + q) + qp^2 N^2)$. The algorithm of triple exponential smoothing runs in time $O(1)$.

4 SYSTEM DESIGN AND IMPLEMENTATION

In this section, we first describe the overall design of a container resource load prediction system, then we elaborate the individual modules in detail.

4.1 System Architecture

Fig. 3 shows the overall design of the Docker container resource load real-time prediction system based on a hybrid model of ARIMA and triple exponential smoothing.

The system consists of the following modules: *container information collection*, *prediction container selection*, *container resource acquisition*, *container data storage*, *container resource prediction*, and *container resource scheduling*.

The main functions of these modules are shown below:

- The *container information collection* module is to collect the ID and status of starting and stopping the container through the Docker API. If the container status is *start*, the container is added to the queue of the *prediction container selection* module; if the container status is *stop*, the container is removed from the queue.
- The *prediction container selection* module is to maintain a container queue to be predicted and scheduled according to the next prediction scheduling time of the container. Each time the first container in the queue is monitored, the container ID is acquired, and the *container resource acquisition* module is called for data collection.
- The *container resource acquisition* module obtains the container ID sent by the *prediction container selection* module, collects the CPU usage, memory usage, disk read rate, disk write rate, network receiving rate and network transmission rate of the container, and sends them to the *container data storage* module.
- The *container data storage* module stores the container resource data into the database, and then organizes the data into a specified format and sends it to the *container resource prediction* module.
- The *container resource prediction* module employs the ARIMA-triple exponential smoothing model to predict the resource usage of the container, and sends the prediction result to the *container resource scheduling* module.
- The *container resource scheduling* module dynamically updates the resource usage (CPU and Memory usage) of the container according to the prediction result of *container resource prediction* module.

4.2 Container Information Collection

The *container information collection* module collects the container start, stop, ID, image and task information through the Docker API. This module maintains the state list called ContainerInfoList of each container that is running. The elements in ContainerInfoList are shown in Table 1.

TABLE 1
Element Definition of ContainerInfoList

Field name	Type of data	Description
ContainerImage	string	Image
ContainerTask	string	Task
ContainerID	string	Container ID

We use the “docker events” command to get real-time events about the container that occurs in the host. When we find that a container has started, we use the “docker inspect containerID” command to get the basic information of its image information and task information according to its container ID. Then we store the container’s information in ContainerInfoList. When the container stops running, the related element is deleted from ContainerInfoList based on container ID. In addition, we have to send the container startup and stop information to the *prediction container selection* module in the following format: { “container_id”: ContainerID, “type”: type }. The type has two values: *start* and *stop*.

4.3 Prediction Container Selection

Algorithm 3. Prediction Container Selection

Input: *Container_queue* // *Container_queue* is a queue that stores the container to be predicted and scheduled, and the elements are shown in Table 2.

```

1: while true do
2:   if Container_queue is not empty then
3:     get the first container from Container_queue
4:     next_time ← first_container.nextTime
5:     if current_time < next_time then
6:       sleep_time ← next_time - current_time
7:       sleep(sleep_time)
8:     else
9:       ID ← first_container.ContainerID
10:      first_container.nextTime ← current_time
          + first_container.predictCycle
11:      send the ID to container resource acquisition model
12:      update the Container_queue according to the nextTime
13:    end if
14:  end if
15: end while

```

The *prediction container selection* module maintains a container queue to be predicted and scheduled, and performs incremental sorting according to the next predicted scheduling time of the container. The elements for each container in this queue are shown in Table 2 and the process of prediction container selection is shown in Algorithm 3. According to the container startup and stop information sent by the *container information collection* module, the containers to be predicted are added into the queue or deleted from the queue, and the order of the container queue is updated accordingly. At the same time, a thread is started in the module to select the container to be predicted from the queue. The thread execution process is as follows: if the current container queue is empty, we do nothing but wait. Otherwise, the container ID and the next predicted time of the first container of the queue are taken out, and the next

TABLE 2
The Elements of Each Container in the Queue

Field name	Type of data	Description
ContainerID	string	Container ID
predictCycle	int	Container prediction period
nextTime	timet	Next container prediction time

predicted time is compared with the current time. If the current time does not reach the next predicted time of the container, the current process is blocked until the next predicted time. If the predicted time is reached, the next predicted time of the first container is updated as the current time plus the prediction period. The order of the container queue is then updated according to the prediction time. Finally, the container ID is sent to the *container resource acquisition* module to collect the data of the container.

4.4 Container Resource Acquisition

The resource usage of the container is acquired by reading the data recorded in the cgroups folder on the host machine and the */proc* virtual file system.

(1)CPU usage

To calculate the CPU usage of the container on the host, we first read the CPU time slices the container has used so far from the host’s cgroup file (*/sys/fs/cgroup/cpuacct/docker/containerID/cpuacct.usage*), record it as *cpu_use_time*. Then we read the total CPU time slice from the */proc/stat* directory, denoting it as *cpu_total_time*. After reading the two data, we use the latter to divide the former to get the CPU usage of the container. If the host has multiple processors, multiply the calculated CPU usage by the number of processor cores as follows:

$$U_{cpu} = \frac{cpu_use_time_{t2} - cpu_use_time_{t1}}{cpu_total_time_{t2} - cpu_total_time_{t1}} \times cores \times 100\%.$$

(2)Memory usage

After the container is started, the host allocates a certain amount of memory for it, and the value is stored in the host’s cgroup file (*/sys/fs/cgroup/memory/docker/containerID/memory.limit.in_bytes*) and recorded as *mem-Limit*. There exists another cgroup file (*/sys/fs/cgroup/memory/docker/containerID/memory.usage_in_bytes*) that stores the amount of memory used by the container, which is recorded as *memUsed*. So the calculation formula for memory usage is

$$U_{mem} = \frac{memUsed}{memLimit} \times cores \times 100\%.$$

Using the same method, we can also calculate the disk read and write rates and the network receiving and sending rates.

4.5 Container Data Storage

This module is to receive the container resource data sent by *container resource acquisition* and store it in the database. Then it organizes the data into a specified format and sends it to the *container resource prediction* module. We design two tables in the database, one is the data storage table, and the other is the control table. The database chosen is InfluxDB

[23], an open-source distributed timing, event, and metrics database.

We use the control table to implement the following functions:

- When the program starts, it will start a thread to scan the control table periodically and read the timestamp of the container in the control table. If the container is not updated beyond a specified period of time, the container is treated as a closed container, and we delete the corresponding rows in the control table and the data in the data storage table. This saves the storage overhead of the database.
- When receiving the container resource data sent by the *container resource acquisition* module, we search the container ID in the control table. If the same ID is found, we store the data in the data storage table and then update the control table. Specifically, the timestamp field is updated and the field that stores the number of rows plus one. If the same ID is not found, we create a new row in both the data storage table and the control table.
- After the container's new resource usage data is written, the number of rows of data in the control table is read. If it does not satisfy the number of rows required for the initial data of the *container resource prediction* module (we record this number as m which is equal to the time series window size), it does nothing but wait for the next database write. If it is exactly equal to m , we get the m rows of container resource usage data from the container's data storage table and put them into an array. Then we send the array to the *container resource prediction* module, which makes the next prediction of the container resource usage. If it is greater than m , we just need to send the latest row of the container's data storage table to *container resource prediction* module. The latest row will replace the first row of the array to form a new time series window.

4.6 Container Resource Prediction

This module uses the ARIMA-triple exponential combination model algorithm to predict the container resource. But it needs to perform redundant deletion and missing value padding on the acquired data before making a prediction. This is due to the possibility of data loss or duplication in a large amount of container data acquired. Either way, it will have an impact on the outcome of the prediction. To remove redundant data, it is necessary to refer to the identification of the data in the data set. The identifier used here is the container ID and the acquisition time of the data. If multiple data have the same container ID and the acquisition time, it means that the data is duplicated, and we should delete it. For serious missing data, we need to discard the data; If the missing data is not serious, Lagrange interpolation can be used to complete the missing data.

After the redundant deletion of the data and the missing value padding operation, the acquired time series data set can be predicted. The process of prediction is as follows.

- 1) After each time series of container resource usage is updated, the weighting coefficients of the hybrid

model are also updated using the method described in Section 3.2 at the same time.

- 2) After determining the weighting coefficient, the updated time series of container resource usage is predicted using ARIMA model and triple exponential smoothing model respectively.
- 3) The prediction result of the hybrid model is obtained according to the prediction result of ARIMA model and triple exponential smoothing model, and is sent to the *container resource scheduling* module.
- 4) After collecting the true value of the current period sent by the container data storage module, return to 1) for the next prediction and scheduling.

4.7 Container Resource Scheduling

This module currently implements dynamic update of CPU and memory resources that are allocated to the container. After obtaining the predicted value of the container resource usage, the container resource cannot be directly update with the value. First, the prediction itself may have errors, and second, the data has random volatility. These two factors should be considered when redistributing the resources. For these two factors, the maximum fluctuation value in the time series of container resource usage is represented as the maximum value minus the minimum value in the time series for prediction, so the new distribution value calculation formula is as follows:

$$NewLimit = Predict + 2 \times (Max\{y_t\} - Min\{y_t\}).$$

Where y_t is the time series for prediction, *Predict* is the predicted value of the container resource usage, and *NewLimit* is the assigned value of the container resource usage.

For CPU resource, after *NewLimit* of CPU usage is obtained, we call the *docker update -cpu-period= <value> -cpu-quota= <value> containerID* directive to limit the CPU usage of the container, where *-cpu-period* is the scheduling period for the CPU usage of each container. The default value of *-cpu-period* is 100 ms. *-cpu-quota* is the maximum CPU time that the container can use in the period, and its value is the value of *-cpu-period* multiplied by *NewLimit*.

For memory resource, at the beginning of the creation of each container, the cgroups limits the maximum memory that each container can occupy. It only needs to adjust the value according to the predicted value.

5 EXPERIMENTAL EVALUATION

In this section, we will first describe the experimental environment, then we make sensitive analysis of the parameters of the hybrid model. At last, we compare the hybrid model with a series of workload prediction models from the aspect of prediction accuracy, prediction time, and computational costs.

5.1 Experimental Environment

We perform experiments in both simulated and real cloud environments and the configuration is shown in Table 3. The simulated cloud environment is with 8 CPU cores and 32 GB RAM and runs Ubuntu 16.04 and Docker 18.03.1-ce. For the real cloud environment, we adopt the Amazon EC2 cloud platform. We use two types of instances. One is called t2.micro which is for free with limited use of 1 CPU core

TABLE 3
Configuration Information of the Cloud Environment

Instance	Hardware	Software
simulated	Intel(R) Core(TM) CPU E5620 @3.40GHz 8 Cores, 32GB RAM	Ubuntu 16.04
		Docker 18.03.1-ce
		Memcached 1.5.6
		Stress 1.0.4
t2.micro	Intel(R) Xeon(R) CPU E5-2676 @2.40GHz 1 Cores, 1GB RAM	Postmark 1.51
		Ubuntu 18.04
		Docker 18.09.7-ce
		Memcached 1.5.6
t2.2xlarge	Intel(R) Xeon(R) CPU E5-2686 @2.30GHz 8 Cores, 32GB RAM	Stress 1.0.4
		Postmark 1.51
		Docker 18.09.7-ce
		Memcached 1.5.6

and 1 GB RAM. Another is called t2.2xlarge with 8 CPU cores and 32 GB RAM. Both of the platforms run Ubuntu 18.04 and Docker 18.09.7-ce.

We use three typical application workloads to evaluate the predicted result of the hybrid model. The first type of workload is the Memcached [24] container, which is a free, open source, high-performance distributed memory object caching system that stores the results from database calls, API calls, or page renderings in memory in a key-value store. For the container, we use Mutilate [25] as the load generator, mainly to consume the CPU resources of the container. By adjusting the parameters of the load generator, the generated load can form a time series with different fluctuations. When the parameter is small, the load of the Memcached container is lighter and more stable. When the parameter is larger, the load of the Memcached container is heavier and the corresponding fluctuation is larger.

The second is the stress tool [26], which is a simple workload generator for POSIX systems. It can impose configurable CPU, memory, I/O, and disk pressure on the system. It is written in C and is free software licensed under the GPLv2. We start ubuntu mirror container, install the Stress in the container, execute the command *Stress - -vm 1 - -vm-bytes 100M - -timeout 3600s*, which means to add a new memory allocation process.

The third is Postmark [27], which is a benchmark used to simulate the behavior of mail servers. It is divided into three phases. In the first phase, a file pool is created; in the second phase, four types of transactions are executed: create, delete, read, and attach files; in the final phase, the file pool is deleted. Since we cannot set the test time for Postmark, we add a loop to the *cli_run* function of the Postmark source code, then compile and run it in the container. We also set the file size, read and write concurrent parameters, collect the resource usage data of the container, and then make prediction of the resource usage.

As each kind of container mainly consumes different resource, we choose to predict CPU usage for the Memcached and memory usage for the Stress and Postmark.

In addition, we use google cluster-usage traces [28] to further evaluate the hybrid model. A google cluster is a set of

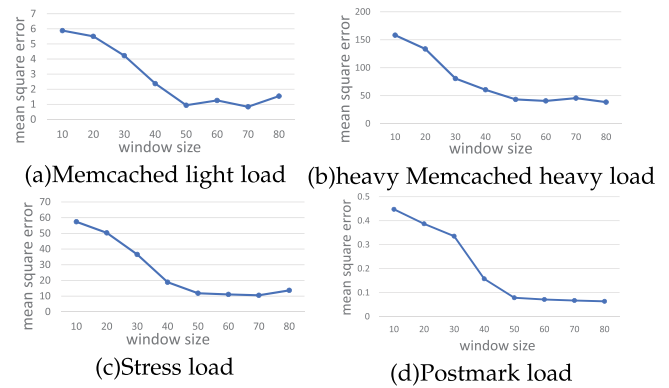


Fig. 4. Mean square error variation of four time series of container resource usage in different window sizes.

machines connected by a high-bandwidth cluster management system that allocates jobs to machines. A job consists of one or more tasks that are accompanied with a set of resource requirements.

5.2 Parameter Selection of Hybrid Model

5.2.1 Time Series Window Size

For real-time prediction, analysis of too much historical data could result in a long prediction time and requires a large amount of space to record historical data. However, if the historical data used is small, the prediction accuracy will be low. Therefore, the reasonable selection of historical data is critical to the prediction system. The following experiments calculate the mean square error after 50 predictions using the time series of the three workloads in different size of window. The result is shown in Fig. 4.

As can be seen from Fig. 4, when the window size is between 10 and 50, as the window size increases, more historical data will be used, and the accuracy of prediction will be improved. However, with the further increase of the window size, the prediction accuracy is not significantly improved. In addition, as more historical data is used, the prediction time is bound to increase. Considering the above, the window size of ARIMA model, triple exponential smoothing model and the hybrid model is 50.

5.2.2 The Smoothing Factor

For the exponential smoothing method, whether the selection of the smoothing factor α is reasonable has a great impact on the prediction accuracy. The smoothing factor determines the corresponding sensitivity to the gap between the predicted value and the actual value, that is, the closer the smoothing factor is to 0, the slower the influence of the distant historical data on this prediction declines, and the closer the smoothing factor is to 1, the more rapidly the influence degree of distant historical data on this prediction declines. In addition, the smoothing factor also determines the ability of the model to smooth the random error generated during the prediction process. The larger the smoothing factor, the stronger the smoothing ability. Therefore, the selection of the smoothing factor is the key to exponential smoothing prediction. The experiment selects the optimal smoothing factor for the time series of the three application workloads. The mean square errors of the prediction of the

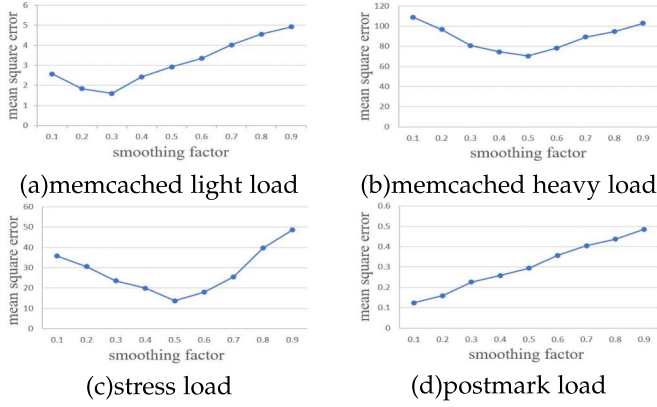


Fig. 5. Mean square error variation of four time series of container resource usage under different smoothing factors.

exponential smoothing method are calculated after 50 times of predictions, and the optimal smoothing factor is selected according to the mean square error.

As can be seen from Fig. 5, for the two large fluctuations of the Memcached heavy load and the Stress load, optimal smoothing factors are both 0.5. The optimal smoothing factor of Memcached light load is 0.3, and the optimal smoothing factor of Postmark is 0.1. This is because when the data fluctuates greatly, we generally choose a larger smoothing factor, increase the weight of the recent data, and also smooth the data. When the data fluctuation is small, the smoothing factor should be chosen to be smaller. The smoothing factor of the Postmark load is smaller than the smoothing factor of the Memcached light load, indicating that the resource usage of the Postmark is more stable than it.

5.3 Comparison of Prediction Accuracy

We compare the prediction accuracy of four prediction models. They are ARIMA model, triple exponential smoothing model, the hybrid model and a workload prediction model using neural network and self adaptive differential evolution algorithm [29] which we call ANN+SaDE model. ANN+SaDE model uses genetic algorithm to train neural network to make prediction of time series.

The evaluation of prediction performance is based on mean absolute percentage error(MAPE) and mean squared error(MSE) which are widely used error metrics for evaluating results of time-series prediction, their formulas is shown as blow:

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{A_i - P_i}{A_i} \right| \quad (9)$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (A_i - P_i)^2. \quad (10)$$

Where A_i is the actual value and P_i is the predicted value.

5.3.1 Experiment in Local Simulated Cloud Environment

We first compare the predicted results of the four models under different loads in local simulated cloud environment. The predicted results are shown in Fig. 6 and Table 4. In Fig. 6, the first 50 points in the ARIMA, triple exponential smoothing model and the hybrid model have no prediction curve. This is because the initial window size is 50, and these 50 data are historical data used by

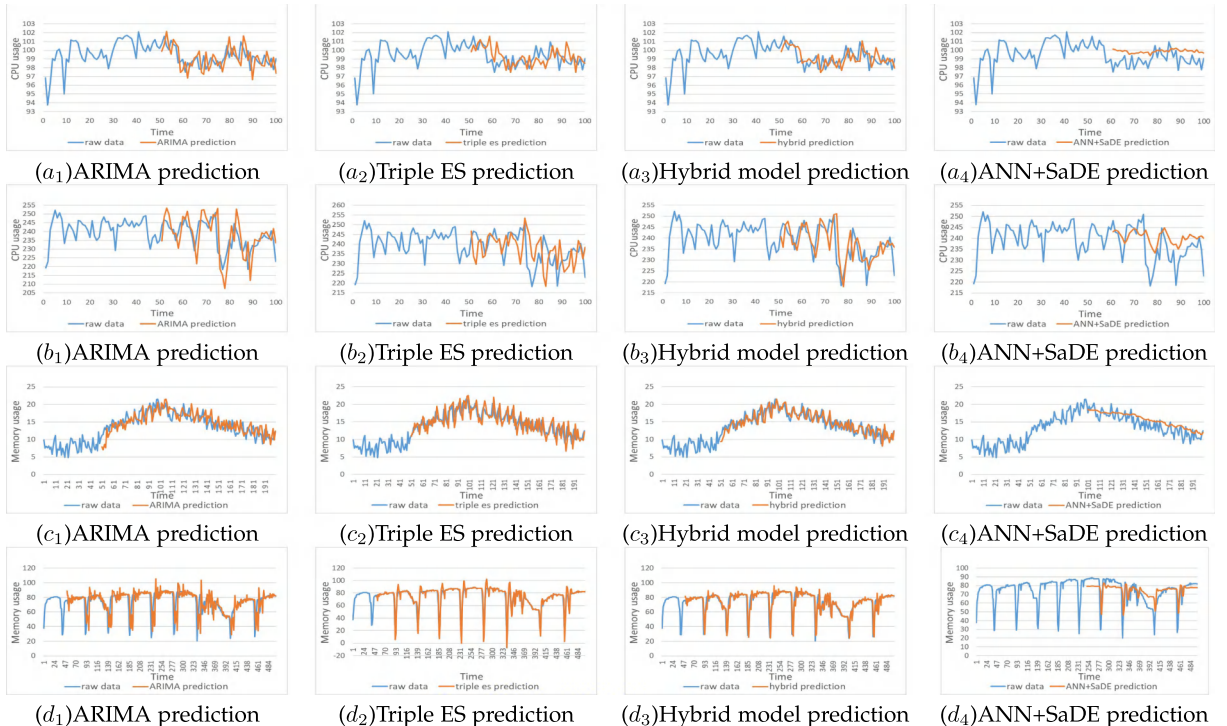


Fig. 6. The prediction effect diagram of four models under different loads in simulated cloud environment. $(a_1 - a_4)$ represents the prediction effect of Memcached light load, $(b_1 - b_4)$ represents the prediction effect of Memcached heavy load, $(c_1 - c_4)$ represents the prediction effect of Stress load, $(d_1 - d_4)$ represents the prediction effect of Postmark load. The blue line shows the real CPU or memory usage, and the red line shows the prediction result of the different models.

TABLE 4
Prediction Result in Simulated Cloud Environment

the load type	prediction model	MAPE	MSE
Memcached light load	ARIMA	0.897%	1.238
	triple exponential smoothing	1.062%	1.595
	hybrid model	0.805%	0.935
	ANN+SaDE	0.934%	1.242
Memcached heavy load	ARIMA	3.126%	91.340
	triple exponential smoothing	2.878%	70.330
	hybrid model	1.987%	43.262
	ANN+SaDE	2.433%	54.703
Stress load	ARIMA	11.285%	3.447
	triple exponential smoothing	13.932%	5.634
	hybrid model	10.276%	2.902
	ANN+SaDE	11.731%	3.484
Postmark load	ARIMA	12.415%	153.841
	triple exponential smoothing	7.252%	54.781
	hybrid model	7.216%	49.187
	ANN+SaDE	13.162%	102.542

the models for the first prediction. There are more points that have no prediction curve for ANN+SaDE. This is because the neural network needs to be trained first, and thus ANN+SaDE needs more data for training than the other three models.

The prediction accuracy of the hybrid model is better than both of the single models. This is because the hybrid model will give more weight to the model with small prediction deviation for different time series. In other word, the hybrid model is able to combine the advantages of the two models to a certain extent. It mines more useful information of time series, so the prediction accuracy is improved.

Compared with the ANN+SaDE model, the prediction accuracy of the hybrid model is also better. This is because the resource time series generated by the container load is real-time, and the trend of the previous training data for ANN+SaDE model will not be the same as the trend of the prediction data.

5.3.2 Experiment in Amazon Elastic Compute Cloud

Considering the heterogeneity between the local physical machine and the real cloud environment, we conducted further experiments on the prediction accuracy of the four models using Amazon EC2 cloud platform as shown in

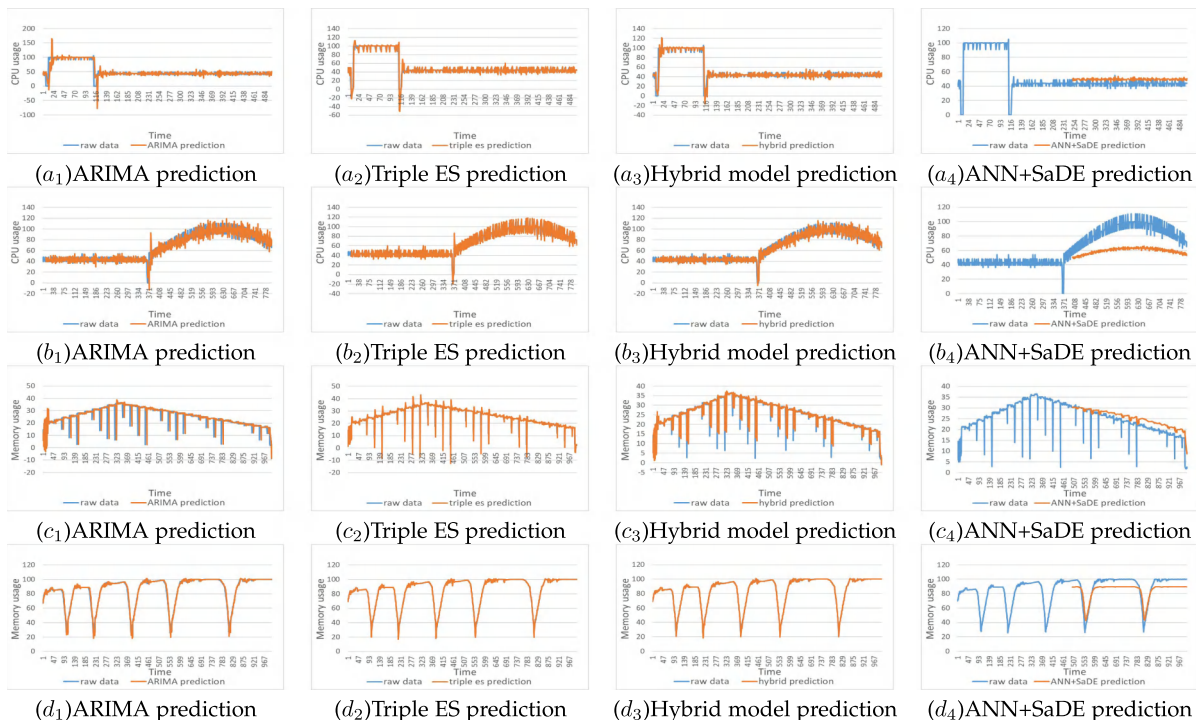


Fig. 7. The prediction effect diagram of four models under different loads in t2.micro instance. ($a_1 - a_4$) represents the prediction effect of Memcached light load, ($b_1 - b_4$) represents the prediction effect of Memcached heavy load, ($c_1 - c_4$) represents the prediction effect of Stress load, ($d_1 - d_4$) represents the prediction effect of Postmark load. The blue line shows the real CPU or memory usage, and the red line shows the prediction result of the different models.

TABLE 5
Prediction Result in t2.micro Instance Environment

the load type	prediction model	MAPE	MSE
Memcached light load	ARIMA	10.241%	33.041
	triple exponential smoothing	12.917%	52.693
	hybrid model	10.153%	32.346
	ANN+SaDE	14.372%	48.814
Memcached heavy load	ARIMA	10.265%	122.575
	triple exponential smoothing	11.954%	174.664
	hybrid model	10.039%	116.503
	ANN+SaDE	30.804%	866.069
Stress load	ARIMA	14.026%	9.619
	triple exponential smoothing	9.876%	13.528
	hybrid model	9.497%	7.998
	ANN+SaDE	24.604%	13.762
Postmark load	ARIMA	2.078%	5.544
	triple exponential smoothing	0.767%	1.120
	hybrid model	0.696%	0.992
	ANN+SaDE	10.920%	92.338

Table 3. We use t2.micro and t2.xlarge instances to measure the cases of single core and multi-core respectively.

The experimental results for the t2.micro instance are shown in Fig. 7 and Table 5. In order to simulate a sudden traffic request, we adjust the load generator's $-T$ parameter to add more CPU overhead at a certain moment in Memcached heavy load. And we adjust memory usage through the $-vm-bytes$ parameter in the Stress load to make the trend of resource usage time series go up and then down. Under all loads, the prediction accuracy of the hybrid model is also the highest. The prediction accuracy of the ANN

+SaDE model is much lower than the other three models. This is because the neural network is trained using the initial part of historical data, and its prediction accuracy will greatly decrease when the time series trend changes, and the trends of different resource time series in Amazon cloud environments change more than that in simulated cloud environments.

We perform further experiments on the t2.xlarge instance with 8 CPU cores to compare the prediction accuracy of different models in a higher physical configuration environment. The experimental results are shown in Fig. 8

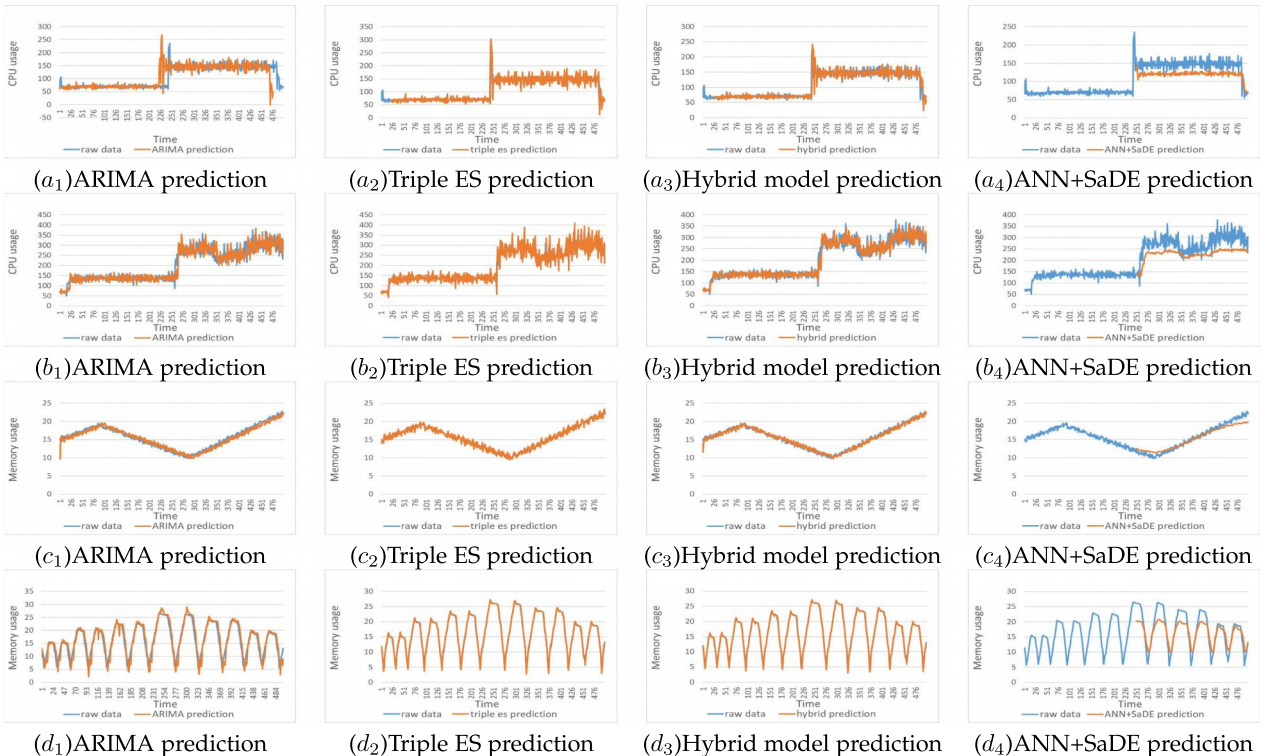


Fig. 8. The prediction effect diagram of four models under different loads in t2.xlarge instance. ($a_1 - a_4$) represents the prediction effect of Memcached light load, ($b_1 - b_4$) represents the prediction effect of Memcached heavy load, ($c_1 - c_4$) represents the prediction effect of Stress load, ($d_1 - d_4$) represents the prediction effect of Postmark load. The blue line shows the real CPU or memory usage, and the red line shows the prediction result of the different models.

TABLE 6
Prediction Result in t2.xlarge Instance Environment

the load type	prediction model	MAPE	MSE
Memcached light load	ARIMA	12.469%	567.937
	triple exponential smoothing	13.184%	540.428
	hybrid model	11.140%	365.493
	ANN+SaDE	17.719%	843.932
Memcached heavy load	ARIMA	10.855%	1304.671
	triple exponential smoothing	14.592%	2529.447
	hybrid model	10.233%	1180.015
	ANN+SaDE	16.672%	3086.117
Stress load	ARIMA	3.819%	0.504
	triple exponential smoothing	4.013%	0.512
	hybrid model	3.120%	0.323
	ANN+SaDE	5.329%	1.069
Postmark load	ARIMA	12.720%	4.127
	triple exponential smoothing	4.240%	0.538
	hybrid model	3.929%	0.499
	ANN+SaDE	15.253%	8.674

and Table 6. Similar as in t2.micro instance, the hybrid model has the highest prediction accuracy, and the ANN+SaDE model has the lowest prediction accuracy.

5.3.3 Experiment in Google Cluster-Usage Traces

The trace we choose is Google cluster-usage traces *cluster-data-2011-2*. We randomly select a long duration job with jobID 3418309 and choose task index 0 and task index 1 in the job. The experimental results are shown in Fig. 9 and Table 7. From Fig. 9, we can see that performance trend of the two traces fluctuates slightly and the tasks consume very little CPU resource. Table 7 shows that the hybrid model has the better prediction accuracy than the ANN+SaDE model. This is because the ANN+SaDE model is

trained using the first 40 percent of the traces which as a whole is trending downward, resulting in its prediction value being greater than the actual value. The hybrid model uses the recent historical data to make predictions, so it can find the data change trend more quickly. Thus it has better prediction results.

5.4 Prediction Time

For the real-time prediction model, the prediction time should not be too long. A long prediction time may cause the current prediction result come out after the actual value of the next period has been collected, so the prediction itself loses its meaning. For the hybrid model based on ARIMA model and triple exponential smoothing model, the main

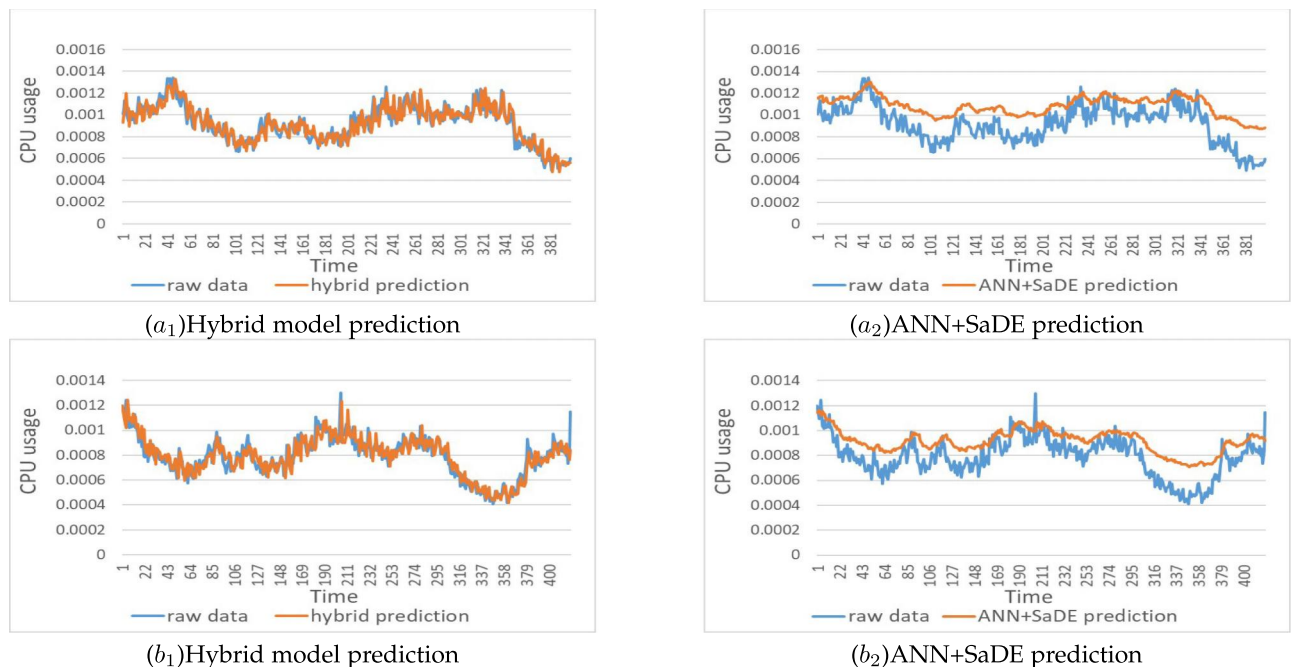


Fig. 9. The prediction effect diagram of four models on Google cluster data. a_1 and a_2 represent the prediction effect of task index 0, b_1 and b_2 represent the prediction effect of task index 1. The blue line shows the real CPU usage, and the red line shows the prediction result of the different models.

TABLE 7
Prediction Result in Google Cluster Data

the load type	prediction model	MAPE	MSE
task index 0	hybrid model	6.857%	6.41e-09
	ANN+SaDE	20.581%	3.51e-08
task index 1	hybrid model	7.388%	5.914e-09
	ANN+SaDE	19.165%	2.23e-08

prediction time consumption lies in ARIMA model. Because each prediction of triple exponential smoothing model is based on the previous prediction. Although the model considers all historical data, it does not require to record and use all historical data. Thus the prediction time for triple exponential smoothing is quite short and almost negligible. For ARIMA model and the hybrid model, 50 predictions are performed under different loads, and their average prediction time is shown in Fig. 11.

The prediction time of the hybrid model is slightly longer than that of ARIMA model by 0.59, 5.4, 3.17 and 1.3 percent in the four-time series respectively. The extra time overhead is almost negligible because the time overhead of triple exponential smoothing model is small.

5.5 CPU and Memory Overhead

We test the CPU and memory overhead of running the hybrid and ANN+SaDE models using the google cluster data as shown in Fig. 10. We remove the prediction interval from the programs and record the CPU and memory usage every 0.01 seconds. Because the execution time of the hybrid model is much less than ANN+SaDE model, the curve of the hybrid model is much shorter than the ANN+SaDE model. The experimental result shows the CPU usage of the hybrid model is lower than the ANN+SaDE model. This means the computational cost of the hybrid model is lower than ANN+SaDE model. In addition, we can see from the figure that the memory usage of the hybrid model is also much lower than the ANN+SaDE model. This is because artificial neural network training consumes a lot of CPU and memory resources.

6 RELATED WORK

We first introduce the general prediction method and then describe the related work on Docker container resource load prediction.

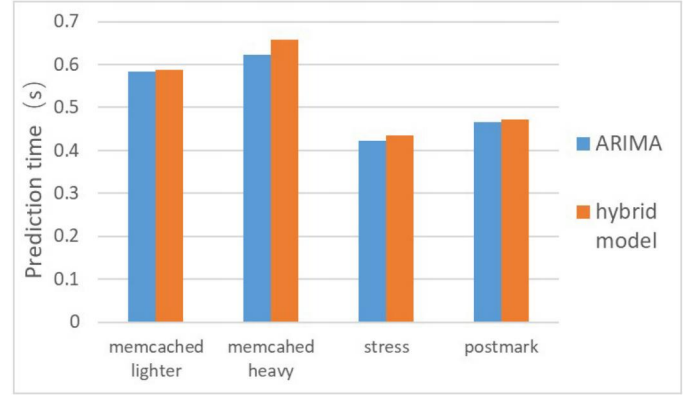
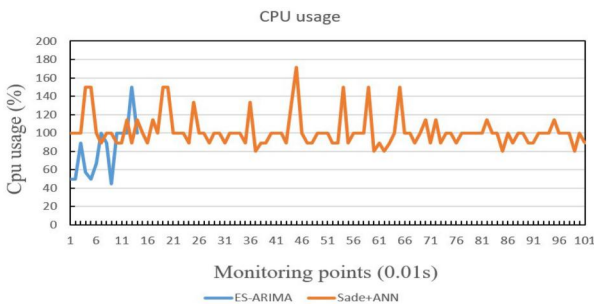


Fig. 11. Average prediction time comparison between the two models.

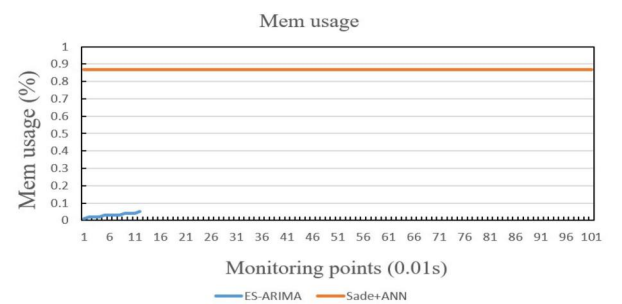
6.1 Prediction Method

The trend extrapolation is a technique of predicting using statistical methods in order to predict future patterns of time series data [30]. The trend extrapolation can be subdivided into two types: the moving average method, and the exponential smoothing method. The moving average is extremely useful for forecasting long-term trends. When the value of the time series is affected by the periodic variation and random interference, the fluctuation of the time series is large, and the development trend of the time series cannot be clearly displayed. The moving average method can effectively eliminate the influence of the random factor and explore the overall trend of the time series. The moving average method usually does not consider the historical data of a long time, and the full-motion moving average rule uses all the historical data of the time series equally (i.e., gives all the historical data the same weight to calculate the average). The exponential smoothing method combines the advantages of full-motion moving average and moving average. It uses all historical data, and assigns the weight of the historical data from near to far to a value that gradually converges to zero.

Grayscale prediction is a method of predicting gray systems. The process of grayscale prediction is generally to accumulate data first to eliminate the randomness and volatility of the data. Then the white differential equation is established, and the solution of the equation is the prediction result. Khalid *et al.* [31] used the GM(1,1) model in the grey model to predict wind power in a short period of time. The gray prediction model is an exponential prediction



a. CPU consumption



b. Memory consumption

Fig. 10. The resource consumption of host.

model, which is suitable for time series in which the data trend is exponential [32]. But for other trends, the prediction effect may not be so good.

In recent years, SVM [33] has been able to perform high-precision fitting because of its excellent generalization ability, and it has also been applied to solve the problem of regression. However, the selection of model parameters is the difficulty of SVM regression prediction. So far, there is no unified guiding theory. This hinders the development of SVM for data prediction.

The neural network model [34] is an information processing system simulating the structure and function of human brain nerve cells. In fact, it is a complex network composed of a large number of neurons. Each neuron represents an output function, and the connections between neurons represent weights. With different weights and output functions, the final output of the network will be different. Neural networks have very powerful learning capabilities and can learn to approach any nonlinear mapping relationship. Thus they are widely used in various fields of forecasting, such as stock price forecasting in the financial sector [35], and traffic wandering in the transportation sector [36].

6.2 Docker Resource Load Prediction

At present, there are few related studies on resource usage prediction of Docker containers. Shanmugam *et al.* [4] predicted the CPU usage of the container through the ARIMA model, and then distributed the load to the container's web service using a loop-based algorithm.

There are many studies on cloud computing resource load prediction. The dynamic and real-time characteristics of Docker container resources are consistent with the characteristics of cloud computing load. Therefore, researching cloud computing load prediction has a strong reference and learning significance for Docker container resource load prediction.

Calheiros *et al.* [5], [6] proposed a cloud computing load prediction model based on the ARIMA model. First, the time series are smoothed to determine the value of d , and then the values of p and q are determined by the autocorrelation function and the partial autocorrelation function. At this time, the historical data of the load will conform to the determined values of p , d and q . The ARIMA model is then used to predict future load values and can achieve an average of 91 percent accuracy.

Huang *et al.* [7] proposed a resource prediction model based on quadratic exponential smoothing to predict the cloud resources that customers need to subscribe to. It not only considers the current resource status but also considers historical resource records and thus obtains higher prediction accuracy. But the quadratic exponential smoothing model is in essence a linear model and cannot mine the nonlinear relationship in the time series.

Islam *et al.* [8] used a combination of neural networks and linear regression to predict the resources of managed applications in the cloud environment. The method uses traditional linear regression to approximate linear relationships in time series and then uses neural networks to approximate nonlinear relationships in sequences. It takes into account more influencing factors in the time series and obtains better prediction results. However, enough samples

are needed to train the neural network. When applied to the container, it needs to collect a large amount of time series of container resource usage, which has great storage and computation overheads.

7 CONCLUSION

Predicting the resource usage of container workload in dynamic container workload environments has been a great challenge to improve the performance of cloud computing platform. This paper proposes a hybrid model that combines ARIMA with triple exponential smoothing to accurately predict both the linear and non-linear relationships in the time series of the resource workload of Docker container. Besides, to enable automatic prediction and alleviate the management burden, we also design and implement a Docker container resource prediction system that enables efficient container information collection, storage, prediction and schedule. This hybrid model improves prediction accuracy by 52.64, 20.15 and 203.72 percent on average compared to ARIMA, the triple exponential smoothing model and ANN+SaDE respectively with a small time overhead. Users can directly use the system we designed to improve the resource utilization of the container-based cloud platform, or use the hybrid model to implement a resource prediction system adapted to their own platform.

ACKNOWLEDGMENTS

This work was supported in part by the National Science Foundation of China under Grant No. 61972449, U1705261, and 61821003, in part by CCF-NSFOCUS Kun Peng research fund, in part by Wuhan Application Basic Research Program under Grant No. 2017010201010104, in part by Hubei Natural Science and Technology Foundation under Grant No. 2017CFB304, and in part by the Fundamental Research Funds for the Central Universities under Grant No. 2019kfyXKJC021.

REFERENCES

- [1] P. Mell and T. Grance, "The NIST definition of cloud computing," *Commun. ACM*, vol. 53, no. 6, pp. 50–50, 2011.
- [2] K.-T. Seo, H.-S. Hwang, I.-Y. Moon, O.-Y. Kwon, and B.-J. Kim, "Performance comparison analysis of Linux container and virtual machine for building cloud," *Adv. Sci. Technol. Lett.*, vol. 66, no. 2, pp. 105–111, 2014.
- [3] C. Anderson, "Docker [software engineering]," *IEEE Softw.*, vol. 32, no. 3, pp. 102–c3, May/Jun. 2015.
- [4] A. S. Shanmugam, "Docker container reactive scalability and prediction of CPU utilization based on proactive modelling," Masters thesis, Dublin, Nat. College Ireland, 2017. [Online]. Available: <http://trap.ncirl.ie/2884/1/aravindsamyshanmugam.pdf>
- [5] N. Roy, A. Dubey, and A. S. Gokhale, "Efficient autoscaling in the cloud using predictive models for workload forecasting," in *Proc. IEEE Int. Conf. Cloud Comput.*, 2011, pp. 500–507.
- [6] V. G. Tran, V. Debusschere, and S. Bacha, "Hourly server workload forecasting up to 168 hours ahead using seasonal ARIMA model," in *Proc. IEEE Int. Conf. Ind. Technol.*, 2012, pp. 1127–1131.
- [7] J. Huang, C. Li, and Y. Jie, "Resource prediction based on double exponential smoothing in cloud computing," in *Proc. 2nd Int. Conf. Consum. Electron. Commun. Netw.*, 2012, pp. 2056–2060.
- [8] S. Islam, J. Keung, K. Lee, and A. Liu, "Empirical prediction models for adaptive resource provisioning in the cloud," *Future Gener. Comput. Syst.*, vol. 28, no. 1, pp. 155–162, 2012.
- [9] Z. Zou, Y. Xie, K. Huang, G. Xu, D. Feng, and D. Long, "A docker container anomaly monitoring system based on optimized isolation forest," *IEEE Trans. Cloud Comput.*, to be published, doi: 10.1109/TCC.2019.2935724.

- [10] Z. Huang, S. Wu, S. Jiang, and H. Jin, "FastBuild: Accelerating docker image building for efficient development and deployment of container," in *Proc. 35th Symp. Mass Storage Syst. Technol.*, 2019, pp. 28–37.
- [11] LXC. 2013. [Online]. Available: <https://linuxcontainers.org/>
- [12] Libcontainer. 2013. [Online]. Available: <https://github.com/docker/libcontainer>
- [13] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, 2014, Art. no. 2.
- [14] K. Kaur, T. Dhand, N. Kumar, and S. Zeadally, "Container-as-a-service at the edge: Trade-off between energy efficiency and service availability at fog nano data centers," *IEEE Wireless Commun.*, vol. 24, no. 3, pp. 48–56, Jun. 2017.
- [15] N. Ferry, A. Rossini, F. Chauvel, B. Morin, and A. Solberg, "Towards model-driven provisioning, deployment, monitoring, and adaptation of multi-cloud systems," in *Proc. IEEE 6th Int. Conf. Cloud Comput.*, 2013, pp. 887–894.
- [16] N. Naik, "Migrating from virtualization to dockerization in the cloud: Simulation and evaluation of distributed systems," in *Proc. IEEE 10th Int. Symp. Maintenance Evol. Service-Oriented Cloud-Based Environ.*, 2016, pp. 1–8.
- [17] Autoregressive Integrated Moving Average model. 2004. [Online]. Available: <https://people.duke.edu/~rnau/411arim.htm>
- [18] B. Li, J. Zhang, Y. He, and Y. Wang, "Short-term load-forecasting method based on wavelet decomposition with second-order gray neural network model combined with ADF test," *IEEE Access*, vol. 5, pp. 16 324–16 331, 2017.
- [19] K. Yamaoka, T. Nakagawa, and T. Uno, "Application of Akaike's information criterion (AIC) in the evaluation of linear pharmacokinetic equations," *J. Pharmacokinetics Biopharmaceutics*, vol. 6, no. 2, pp. 165–175, 1978.
- [20] Levinson recursion. 2004. [Online]. Available: https://en.wikipedia.org/wiki/Levinson_recursion
- [21] E. S. Gardner Jr, "Exponential smoothing: The state of the art," *J. Forecasting*, vol. 4, no. 1, pp. 1–28, 1985.
- [22] P. S. Kalekar, "Time series forecasting using holt-winters exponential smoothing," Kanwal Rekhi School of Information Technology, 4329008, 2014. [Online]. Available: https://www.researchgate.net/publication/268340653_Time_series_Forecasting_using_Holt-Winters_Exponential_Smoothing
- [23] InfluxDB. 2014. [Online]. Available: <https://www.infoq.com/fr/presentations/influx-db/>
- [24] What is Memcached. 2007. [Online]. Available: <http://memcached.org/>
- [25] Leverich.Mutilate. 2018. [Online]. Available: <https://github.com/leverich/mutilate>
- [26] Stress. 2017. [Online]. Available: https://www.archlinux.org/packages/community/x86_64/stress/
- [27] Postmark. 2006. [Online]. Available: <http://www.filesystems.org/docs/auto-pilot/Postmark.html>
- [28] C. Reiss, J. Wilkes, and J. L. Hellerstein, "Google cluster-usage traces: Format + schema," Google Inc., Mountain View, CA, USA, Technical Report, revised 2014–11–17 for version 2.1, Nov. 2011. [Online]. Available: <https://github.com/google/cluster-data>
- [29] J. Kumar and A. K. Singh, "Workload prediction in cloud using artificial neural network and adaptive differential evolution," *Future Gener. Comput. Syst.*, vol. 81, pp. 41–52, 2018.
- [30] Trend extrapolation. 2017. [Online]. Available: <https://thelawdictionary.org/trend-extrapolation/>
- [31] M. Khalid and A. V. Savkin, "A method for short-term wind power prediction with multiple observation points," *IEEE Trans. Power Syst.*, vol. 27, no. 2, pp. 579–586, May 2012.
- [32] E. Kayacan, B. Ulutas, and O. Kaynak, "Grey system theory-based models in time series prediction," *Expert Syst. Appl.*, vol. 37, no. 2, pp. 1784–1789, 2010.
- [33] Support Vector Machine. 2017. [Online]. Available: <https://www.sciencedirect.com/topics/neuroscience/support-vector-machine>
- [34] Neural Networks Model. 2014. [Online]. Available: https://www.ibm.com/support/knowledgecenter/en/SS3RA7_15.0.0/com.ibm.spss.modeler.help/neuralnet_model.htm
- [35] M. Qiu, Y. Song, and F. Akagi, "Application of artificial neural network for the prediction of stock market returns: The case of the Japanese stock market," *Chaos Solitons Fractals*, vol. 85, pp. 1–7, 2016.
- [36] K. Kumar, M. Parida, and V. K. Katiyar, "Short term traffic flow prediction in heterogeneous condition using artificial neural network," *Transport*, vol. 30, no. 4, pp. 397–405, 2015.



Yulai Xie (Member, IEEE) received the BE and PhD degrees in computer science from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2007 and 2013, respectively. He was a visiting scholar with the University of California, Santa Cruz, in 2010 and a visiting scholar with the Chinese University of Hong Kong, in 2015. He is currently an associate professor with the School of Cyber Science and Engineering, HUST, China. His research interests mainly include cloud storage and virtualization, digital provenance, intrusion detection, machine learning, and computer architecture.



Minpeng Jin received the BE degree in computer science from Northeastern University, Shenyang, China, in 2019. He is currently working toward the master's degree at the Huazhong University of Science and Technology (HUST), Wuhan, China. His research interests include Docker container and virtualization.



Zhuping Zou received the BE degree in computer science from the Central South University of Forestry and Technology, Changsha, China, in 2017, and the master's degree from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2019, respectively. His research interests include Docker container and virtualization.



Gongming Xu received the BE degree in computer science from the Wuhan Institute of Technology, Wuhan, China, in 2018. He is currently working toward the master's degree at the Huazhong University of Science and Technology (HUST), Wuhan, China.



Dan Feng (Member, IEEE) received the BE, ME, and PhD degrees in computer science and technology from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 1991, 1994, and 1997, respectively. She is currently a professor and director of Data Storage System Division, Wuhan National Lab for Optoelectronics. She is also dean of the School of Computer Science and Technology, HUST. Her research interests include computer architecture, massive storage systems, parallel file systems, disk array, and solid state disk. She has more than 100 publications in journals and international conferences, including FAST, USENIX ATC, ICDSC, HPDC, SC, the *Information, Communication & Society*, and IPDPS. She is a member of ACM.



Wenmao Liu received the PhD degree in information security from the Harbin Institute of Technology, Harbin, China, in 2013. He is the director of the Innovation Center of NSFOCUS. After completion of his degree, he served as a researcher with NSFOCUS Inc. During the first two years in NSFOCUS, he was also working with Tsinghua University as a postdoc. His interests are focused on cloud security, IoT security, threat intelligence, and advanced security analytics. He has published a book *Software-Defined Security*, in the next generation inspired by SDN/NFV technology and participate cloud security related national and industrial standards. Now, he has been promoting the adoption of container security, and DevSecOps.



Darrell Long (Fellow, IEEE) received the BS degree in computer science from San Diego State University, San Diego, California, and the MS and PhD degrees from the University of California, San Diego, California. He is distinguished professor of computer engineering with the University of California, Santa Cruz. He holds the Kumar Malavalli endowed chair of Storage Systems Research and is director of the Storage Systems Research Center. His current research interests include the storage systems area include high performance storage systems, archival storage systems, and energy-efficient storage systems. His research also includes computer system reliability, video-on-demand, applied machine learning, mobile computing, and cyber security. He is fellow of the American Association for the Advancement of Science (AAAS).

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**